06
THE ASTRAL CARTOGRAPHER

INTERACTIVE CODEBASE ARCHITECTURE & AST EXPLORER

Massive software repositories resemble sprawling ancient labyrinths. Unpack this complexity by parsing git repositories, analyzing Abstract Syntax Trees (ASTs), and rendering an interactive 3D/2D topological dependency atlas of files, functions, and architecture.

OBJECTIVE

Construct a repository intelligence engine that ingests GitHub repositories, parses source code into an interactive dependency graph, computes architectural health metrics, and recommends contributor onboarding pathways.

FOUNDATIONAL SCOPE (PASS TIER)

- **Repository Ingestion:** Ingest public GitHub URLs or zip archives, parsing directory trees and source files into an indexed structure.
- **Interactive Dependency Atlas:** Force-directed or hierarchical graph visualizing module imports, file linkages, and architectural clusters.
- **Code Exploration Panel:** Side-by-side code viewer displaying file contents, token counts, cyclomatic complexity, and import dependencies.
- **Search & Filter:** Find symbols, functions, and files with instant graph centering and visual dependency highlighting.

ASCENDED MASTERY (VICTORY TIER)

- **Deep AST Parsing:** Tree-sitter / Babel parsing mapping call graphs, function dependencies, and identifying circular import cycles.
- **Architectural Health Scoring:** Code churn vs complexity heatmaps highlighting fragile high-risk 'god files' and technical debt hotspots.
- **Smart Onboarding Pathfinder:** Automated good-first-issue pathway mapping file change blast radius for newcomer pull requests.



DELIVERABLES

- GitHub repository with modular parsing pipeline
- README: AST traversal algorithms & graph rendering optimizations
- Tool disclosure & large repository benchmarking report

TECHNICAL FREEDOM

Cytoscape.js, D3.js, React Flow, or Three.js for visualization. Backend with tree-sitter or language AST parsers in Go/Python/Node. Bonus marks for sub-5s repository parsing and 60fps graph physics on 500+ nodes.